

1. Structured Requirements

1.1 System Objective

Build a **centralized Fleet & Shipment Management System** for moving containerized cargo between **6 warehouses**.

The system must manage the complete lifecycle:

Order → Shipment → Resource Allocation → Origin Scheduling → Yard Entry → Loading → Departure → In-Transit → Destination Yard Entry → Docking → Unloading → Completion

The system must maintain a real-time view of:

- Shipments
 - Trucks
 - Drivers
 - Warehouses
 - Gates
 - Slots
 - Staff
 - Loading/unloading operations
 - Yard occupancy
 - Resource availability
 - Shipment status
 - Truck/driver current warehouse
-

2. Core Domain Model

Warehouse

There are currently **6 warehouses**.

Each warehouse has:

- Warehouse ID
- Location
- Gates
- Slots
- Drivers
- Trucks

- Staff
- Machinery
- Docking/parking area
- Operating hours

Gates

A gate is used for **both loading and unloading**.

Current configuration:

- 2 warehouses → **6 gates each**
- 4 warehouses → **5 gates each**

Example:

Warehouse A

```

|— A1
|— A2
|— A3
|— A4
|— A5
|— A6

```

Warehouse B

```

|— B1
|— B2
|— B3
|— B4
|— B5

```

The system must **not hard-code these numbers**. Gate configuration must be warehouse-specific and configurable.

2.1 Slots

Each gate contains its own available time slots.

Example:

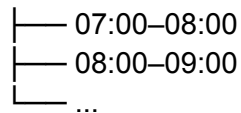
Gate A1

```

|— 07:00–08:00
|— 08:00–09:00
|— 09:00–10:00
|— ...

```

Gate A2



A slot must have:

- Warehouse
 - Gate
 - Start time
 - End time
 - Operation type
 - Loading
 - Unloading
 - Availability
 - Booking status
 - Associated shipment
 - Associated truck
 - Operational status
-

3. Resource Management

3.1 Drivers

Drivers are associated with warehouses based on their **current physical location**.

Example:

Driver A

Initial location → Warehouse A

Shipment:

Warehouse A → Warehouse B

After departure:

Driver A → In Transit

After arrival:

Driver A → Warehouse B

Once Driver A reaches Warehouse B:

- Driver A becomes available at B.

- Driver A should receive higher priority for a shipment from **B** → **A**, because this avoids unnecessary repositioning.

Therefore, the system needs to distinguish:

Home/Assigned Warehouse

Current Warehouse

Current Status

Availability

3.2 Trucks

The same location-management principle applies to trucks.

A truck must maintain:

- Current warehouse
- Current status
- Operational status
- Current shipment
- Availability

Example:

Truck T1

Warehouse A

↓

Departure

↓

IN_TRANSIT

↓

Warehouse B

↓

AVAILABLE

The truck's current location must be updated based on actual operational events.

3.3 Staff

Each warehouse maintains its own staff pool.

Staff **does not need to be permanently assigned to a gate.**

Instead:

Warehouse A
10 available staff

Gate A1 → 3 staff
Gate A2 → 2 staff
Gate A4 → 3 staff
Gate A5 → 2 staff

The system should therefore support **dynamic staff allocation at operation/slot level**.

Staff must support:

- Availability
 - Working status
 - Shift
 - Warehouse
 - Current assignment
-

3.4 Machinery

Machinery belongs to a warehouse and can be allocated to loading/unloading operations.

The allocation model should be similar to staff:

Warehouse
↓
Available machinery
↓
Operation
↓
Slot / Gate

4. Working Hours

Initial business rule:

Warehouse operating time:
07:00 AM onwards

Shift duration:
9 hours

The system must be configurable so operating hours and shifts are not hard-coded.

If a truck arrives after the warehouse operating window:

Truck Arrives



Warehouse Closed



Truck → ON_HOLD



Next Available Operational Window

The shipment should **not be incorrectly marked as failed or completed**.

5. Shipment Creation Flow

The process starts from the **Company UI**.

User enters:

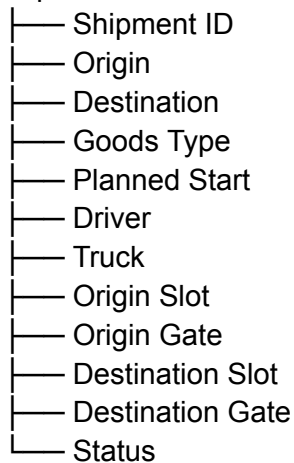
- Origin warehouse
- Destination warehouse
- Goods type
- Shipment/start date
- Required operation timing

System generates:

Shipment ID

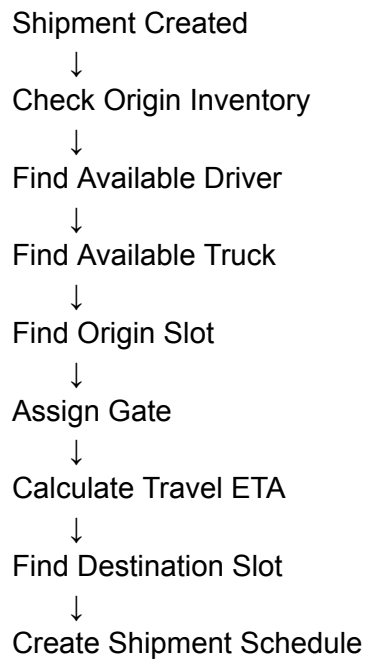
The shipment stores:

Shipment



6. Resource Allocation Flow

Once the shipment is created:



The destination slot must be planned using the estimated arrival time.

7. ETA and Location Management

The system needs warehouse location information.

Example:

Warehouse A
Latitude / Longitude

Warehouse B
Latitude / Longitude

Travel duration between warehouses is required to calculate:

Departure Time
+
Expected Travel Duration
=
Destination ETA

Destination slot selection should use this ETA.

8. Yard Engine

Each warehouse has **one Yard Engine**.

Current architecture:

6 Warehouses



6 Yard Engines

The Yard Engine handles physical truck movement.

Its responsibility is intentionally narrow:

1. Truck Entry

When an expected truck reaches the warehouse:

Expected



Arrived

2. Truck Docking

Truck is moved into the docking/parking area:

Arrived



Docked

3. Truck Exit

When the truck leaves:

Docked



Departed

The yard system must capture actual timestamps for these events.

9. Yard UI

Each warehouse has its own Yard UI.

Operator selects:

Yard

- ☐ Warehouse A
- ☐ Warehouse B
- ☐ Warehouse C
- ☐ Warehouse D
- ☐ Warehouse E
- ☐ Warehouse F

The Yard UI should show:

Current Trucks

Truck

Shipment

Expected Operation

Gate

Slot

Status

Upcoming Trucks

Trucks expected to arrive at the warehouse.

Actions

Mark Arrived

Mark Docked

Mark Departed

The operator should not be responsible for broader shipment/resource management unless explicitly required later.

10. Company Landing Page

The landing page contains the primary user journey.

Create Order

Origin Warehouse [Dropdown]

Destination Warehouse [Dropdown]

Goods Type [Selection]

Start Date/Time [Input]

[Create Order]

After creation:

Shipment ID: SHP-XXXX

The user should be able to use the ID for tracking.

11. Shipment Tracking

Landing page also provides:

Track Shipment

Shipment ID [_____]

[Track]

The result should show the current shipment state.

Example:

Shipment SHP-001

Warehouse A

- ✓ Scheduled
- ✓ Arrived
- ✓ Loaded
- ✓ Departed



In Transit



Warehouse B

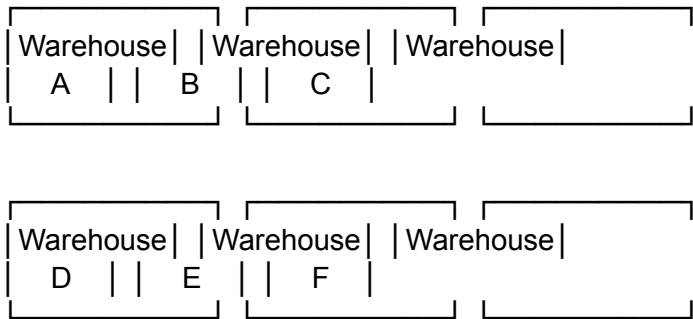
- Expected
 - Arrived
 - Unloaded
 - Completed
-

12. Portal Dashboard

A separate dashboard is available for administrators and warehouse users.

Initial screen:

WAREHOUSE DASHBOARD



Selecting a warehouse opens its operational view.

13. Warehouse Dashboard

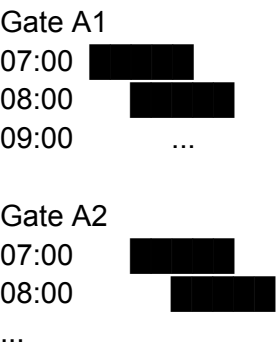
The warehouse dashboard should expose:

Resource Summary

- Drivers
- Trucks
- Staff
- Machinery

Slot Timeline

A timeline/calendar showing:



Each booking must show:

- Shipment

- Truck
 - Operation
 - Gate
 - Slot
 - Status
-

14. TMS / Central Orchestration Layer

The transcript refers to the TMS as the central engine responsible for taking inputs from:

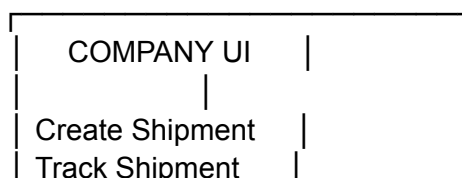
Company UI
+
Yard Engines
↓
Central TMS

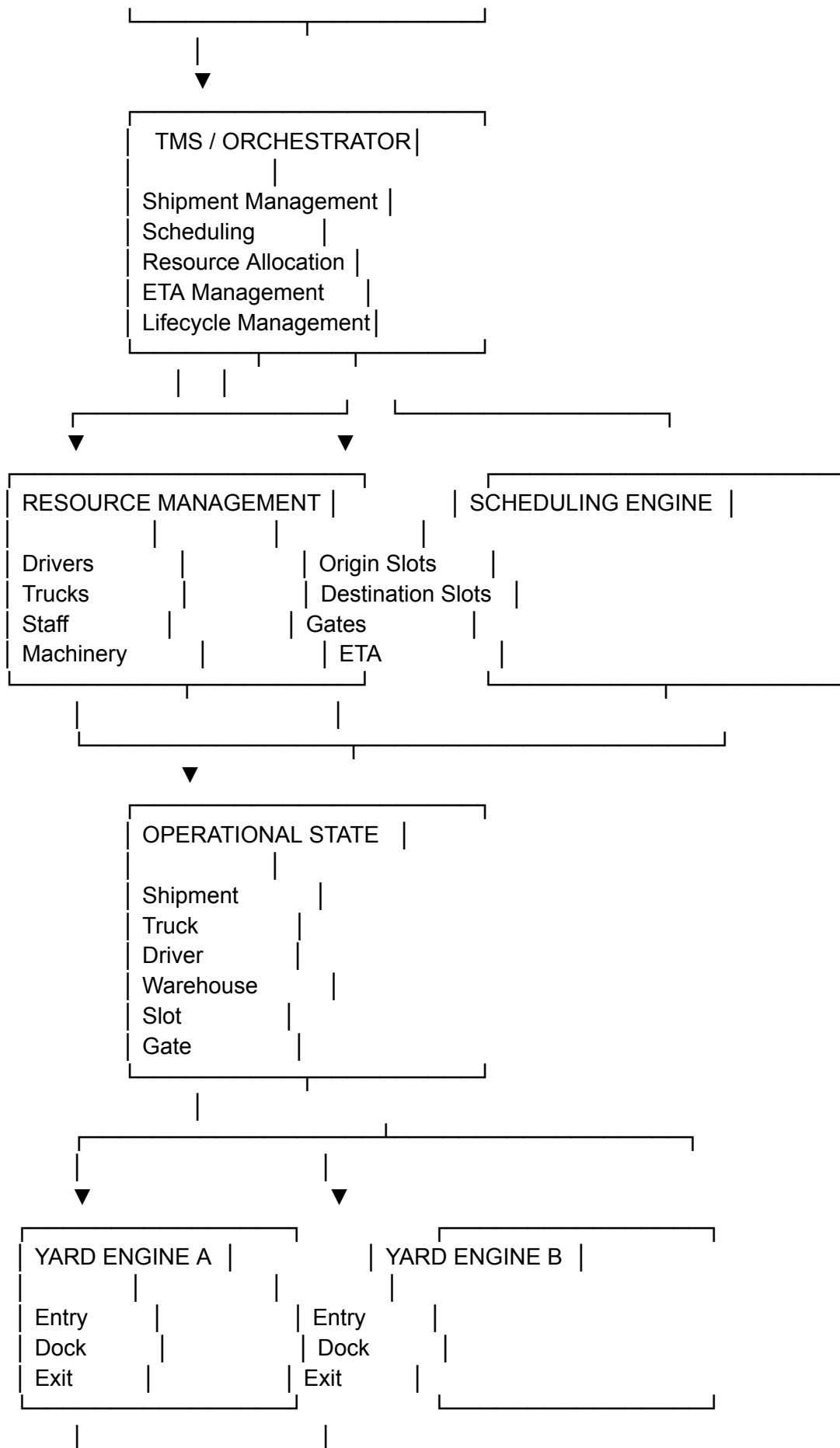
The TMS should therefore act as the **shipment orchestration layer**, rather than directly becoming the physical yard system.

Responsibilities:

- Accept orders
 - Create shipments
 - Maintain shipment lifecycle
 - Allocate resources
 - Schedule origin operations
 - Calculate ETA
 - Schedule destination operations
 - Consume yard events
 - Update shipment status
 - Update truck location
 - Update driver location
 - Coordinate warehouse inventory
 - Expose shipment tracking
-

15. Target Architecture





▼
Warehouse A

▼
Warehouse B

... through Warehouse F

16. Robust Modular Architecture

The system should be designed around **domain boundaries**, rather than individual UI screens.

Recommended modules:

1. Identity & Access
2. Warehouse Management
3. Resource Management
4. Shipment Management
5. Scheduling
6. Yard Operations
7. ETA / Transit Management
8. Tracking
9. Notification / Event Management
10. Reporting & Monitoring

The critical principle is:

UI should represent workflows; domain modules should own business rules.

For example, the Yard UI should not independently modify truck location, shipment status and driver availability. It should submit an operational event, and the central domain logic should apply the resulting state transitions consistently.

17. Detailed Technical Requirement Document

Application Layer 1 — Identity & Access

Purpose

Control who can access which operational functions.

Requirements

- User authentication.
 - Role-based authorization.
 - Support at minimum:
 - Company User
 - Yard Operator
 - Warehouse User
 - Administrator
 - Driver
 - Warehouse-specific access.
 - Yard operator should only operate the assigned yard.
 - Warehouse users should only access authorized warehouses.
 - Administrative users can access all warehouses.
 - Every operational mutation must identify the acting user.
-

Application Layer 2 — Warehouse Management

Purpose

Maintain warehouse configuration and operational state.

Requirements

- Create/manage warehouses.
- Maintain warehouse location.
- Configure operating hours.
- Configure shifts.
- Configure gates.
- Configure gate identifiers.
- Configure slots for each gate.
- Configure docking/parking capacity.
- Configure staff.
- Configure machinery.
- Configure warehouse-specific drivers.
- Configure warehouse-specific trucks.

Business Rules

- Gate must belong to exactly one warehouse.
- Slot must belong to exactly one gate.
- A slot cannot be booked for overlapping operations.

- Warehouse operating hours must influence scheduling.
 - Configuration must support different numbers of gates per warehouse.
-

Application Layer 3 — Resource Management

Driver Management

Requirements:

- Register driver.
- Maintain driver status.
- Maintain current warehouse.
- Maintain current shipment.
- Maintain availability.
- Maintain operational status.
- Assign driver to shipment.
- Release driver after completion.
- Update driver's current location after arrival.
- Support driver prioritization based on current location.

Driver state example

AVAILABLE
ASSIGNED
AT_ORIGIN
IN_TRANSIT
AT_DESTINATION
UNAVAILABLE
NON_OPERATIONAL

Truck Management

Requirements:

- Register truck.
- Maintain current warehouse.
- Maintain current shipment.
- Maintain availability.
- Maintain operational status.
- Assign truck to shipment.
- Release truck after shipment.

- Update current warehouse based on actual arrival.
- Prevent assignment of unavailable trucks.

Truck state

AVAILABLE

ASSIGNED

AT_ORIGIN

LOADING

IN_TRANSIT

AT_DESTINATION

UNAVAILABLE

NON_OPERATIONAL

Staff Management

Requirements:

- Register staff.
 - Assign staff to warehouse.
 - Maintain shift.
 - Maintain availability.
 - Dynamically allocate staff to operations.
 - Release staff after operation.
 - Prevent double allocation.
-

Machinery Management

Requirements:

- Register machinery.
 - Associate machinery with warehouse.
 - Maintain availability.
 - Allocate machinery to loading/unloading operation.
 - Release machinery after operation.
 - Prevent simultaneous conflicting assignments.
-

Application Layer 4 — Shipment Management

Purpose

Own the shipment lifecycle.

Shipment Creation

The system must capture:

- Shipment ID
- Origin warehouse
- Destination warehouse
- Goods type
- Planned start date/time
- Driver
- Truck
- Origin operation
- Destination operation

Requirements

- Validate origin and destination.
 - Prevent invalid origin = destination unless explicitly allowed.
 - Generate unique shipment ID.
 - Maintain shipment lifecycle.
 - Maintain shipment history.
 - Associate all operational resources.
 - Allow shipment tracking.
 - Preserve historical events.
-

Application Layer 5 — Scheduling Engine

Purpose

Allocate warehouse capacity and resources based on time.

Requirements

Given:

Shipment

+

Origin

+
Destination
+
Requested Start

The scheduling engine must:

1. Find available truck.
2. Find eligible driver.
3. Find origin slot.
4. Select gate associated with slot.
5. Determine staff availability.
6. Determine machinery availability.
7. Calculate expected loading completion.
8. Calculate expected departure.
9. Calculate destination ETA.
10. Find destination slot.
11. Reserve destination capacity.

Conflict prevention

The scheduler must prevent:

Same truck

↓

Two shipments at same time

Same driver

↓

Two shipments at same time

Same slot

↓

Two shipments

Same gate

↓

Conflicting operations

Application Layer 6 — Yard Operations

Purpose

Represent actual physical movement of trucks within a warehouse.

Requirements

Yard must support:

Expected Arrival



Truck Arrived



Truck Docked



Truck Departed

Each event must capture:

- Truck
- Shipment
- Warehouse
- Timestamp
- Yard
- Operator
- Gate, where applicable

Yard validation

The system should validate:

- Is the truck expected?
- Is the shipment valid?
- Is the truck assigned to this shipment?
- Is the truck expected at this warehouse?
- Is the truck already inside?
- Can the truck leave?
- Is the required operation completed?

Application Layer 7 — Shipment Lifecycle & State Management

A centralized state machine should govern shipment status.

Example:

CREATED



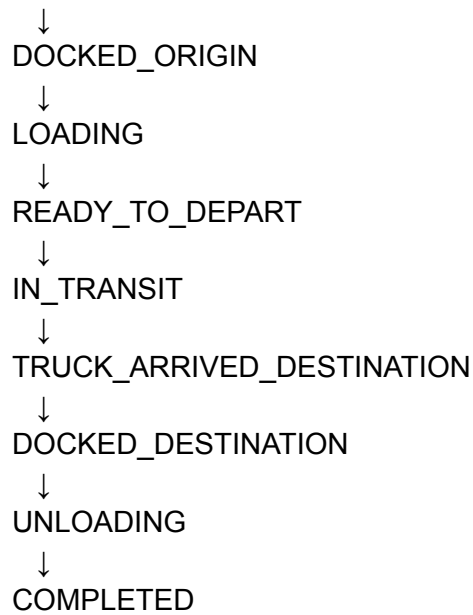
SCHEDULED



TRUCK_EXPECTED



TRUCK_ARRIVED_ORIGIN



Exception states:

ON_HOLD
CANCELLED
DELAYED
FAILED

State changes must occur through valid transitions only.

Application Layer 8 — ETA & Transit Management

Requirements

The system must maintain:

- Origin location
- Destination location
- Planned departure
- Estimated travel duration
- Planned ETA
- Latest ETA
- Actual departure
- Actual arrival

ETA calculation

Origin
+
Destination
+
Travel Duration
+
Departure Time
=
ETA

If ETA changes:

Updated ETA
↓
Check destination slot
↓
Slot still valid?
/ \
Yes No
↓ ↓
Continue Reschedule

This is critical because destination capacity is dependent on arrival time.

Application Layer 9 — Tracking

Purpose

Provide a simple shipment-tracking experience to company users.

Input:

Shipment ID

Output:

- Shipment status
- Origin
- Destination
- Goods type
- Truck
- Current location
- Driver status
- Origin operation status
- Destination operation status

- Planned ETA
 - Latest ETA
 - Actual departure
 - Actual arrival
 - Timeline of events
-

Application Layer 10 — Warehouse Operational Dashboard

The dashboard must provide warehouse-level visibility.

Warehouse overview

Warehouse A

Drivers Available / Assigned / In Transit
Trucks Available / Assigned / In Transit
Staff Available / Assigned
Machinery Available / Assigned

Gate/slot view

Display:

Gate
Slot
Shipment
Truck
Operation
Status

Timeline

Users should be able to understand warehouse capacity visually across time.

Application Layer 11 — Yard Dashboard

Each yard dashboard should display:

Current Yard

- Trucks currently inside.
- Docked trucks.
- Waiting trucks.
- Shipment associated with each truck.
- Expected operation.
- Gate/slot.

Upcoming

- Expected trucks.
- Expected arrival.
- Shipment.
- Planned operation.

Actions

Arrived

Docked

Departed

The interface should make invalid actions unavailable rather than allowing operators to create inconsistent states.

Application Layer 12 — Event & Audit Management

Every important operational change should generate an event.

Examples:

SHIPMENT_CREATED
DRIVER_ASSIGNED
TRUCK_ASSIGNED
ORIGIN_SLOT_BOOKED
DESTINATION_SLOT_BOOKED
TRUCK_ARRIVED
TRUCK_DOCKED
LOADING_STARTED
LOADING_COMPLETED
TRUCK_DEPARTED
ETA_UPDATED
DESTINATION_ARRIVED
UNLOADING_STARTED
UNLOADING_COMPLETED

SHIPMENT_COMPLETED

Each event should retain:

- Event type
- Entity
- Entity ID
- Warehouse
- Timestamp
- Actor/source
- Previous state
- New state
- Relevant operational metadata

This provides traceability and makes the system auditable.

Application Layer 13 — Notification & Exception Management

The system should detect operational exceptions such as:

- Truck arriving late.
- Truck arriving after warehouse operating hours.
- Slot conflict.
- Driver unavailable.
- Truck unavailable.
- Destination slot no longer valid.
- Loading delay.
- Unloading delay.
- Shipment stuck in a state.
- Resource conflict.

The system should create an actionable exception rather than silently changing state.

Application Layer 14 — Reporting & Visibility

The system should eventually support:

Shipment metrics

- Total shipments
- Active shipments
- Completed shipments
- Delayed shipments
- Cancelled shipments

Warehouse metrics

- Gate utilization
- Slot utilization
- Truck throughput
- Average loading time
- Average unloading time
- Yard occupancy

Resource metrics

- Driver utilization
 - Truck utilization
 - Staff utilization
 - Machinery utilization
-

18. Critical Architectural Rules

1. Single Source of Truth

There should be one authoritative operational state for:

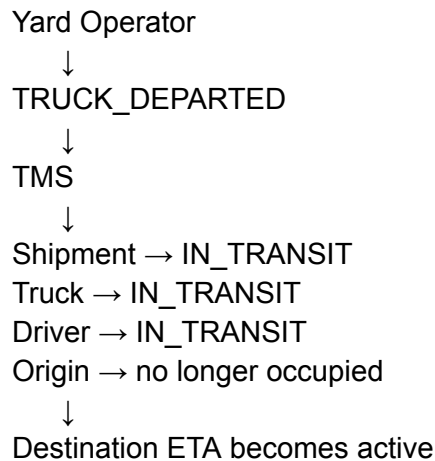
Shipment
Truck
Driver
Warehouse
Slot
Gate

The UI must not maintain competing state.

2. Event-driven operational updates

Yard actions should generate operational events.

For example:



3. Configuration over hard-coding

Do not hard-code:

6 warehouses
5 gates
6 gates
7 AM
9-hour shift

These are **current business configuration**, not architectural constants.

4. Resource location must be explicit

For trucks and drivers, distinguish:

Assigned/Home Warehouse

from:

Current Physical Warehouse

This is essential for the reverse-trip prioritization requirement.

5. Scheduling must be transactional

A shipment should not partially reserve resources.

For example:

Truck reserved
Driver reserved
Slot unavailable

should not leave the truck and driver permanently reserved.

The allocation process needs an atomic reservation/rollback model.

6. Historical state must never be lost

Current state answers:

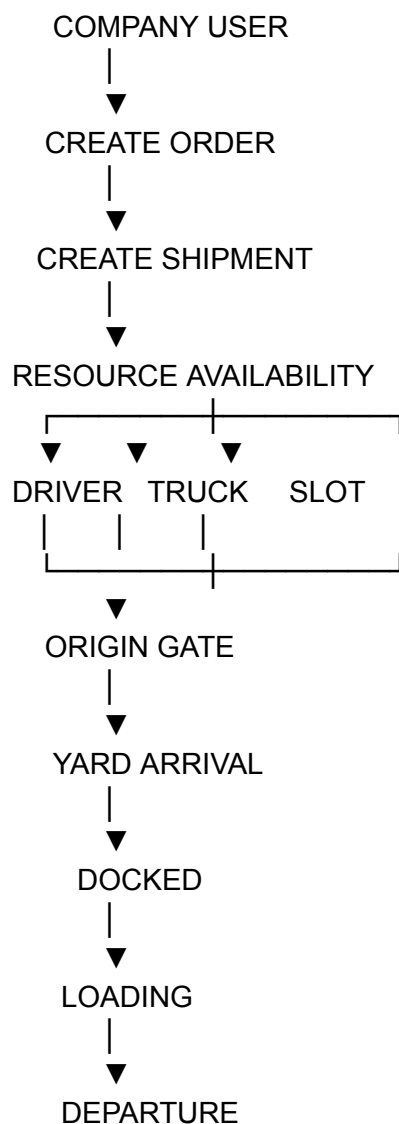
Where is the truck now?

Event history answers:

How did the truck get here?

Both are required.

19. End-to-End Operational Flow





The resulting system should therefore be treated as a **shipment orchestration platform with warehouse scheduling and yard execution**, rather than simply a CRUD-based fleet application. The core architectural challenge is maintaining **consistent real-time state across shipments, resources, warehouse capacity and physical yard events**, while allowing each functional area to evolve independently.